Concepts

New Concepts for this assignment:

1. Using Primitive Arrays (Performance + Speed) ⭐

```
int[] arr = new int[n];
long[] arr = new long[n];
```

Why needed

- Faster than ArrayList
- Required for large inputs (up to 10^6)

---

2. Correct Use of `long` (VERY IMPORTANT) ⭐

```
long sum = a + b;
long currentSum = 0;
```

Why needed

- Values up to 10^9 → int may overflow
- Subarray sums can exceed int

Rule → **Whenever doing addition, use long**

When long doesn't work

```
long n = 10;
long[] arr = new long[n];     // ❌ array size must be int, not long

for(long i = 0; i < n; i++)  // ⚠ Legal but bad practice
```

---

3. Two-Pointer Variables

```
int left = 0;
int right = n - 1;

while (left < right) {
      //Code
       left++;
       right--;
```

Why needed

1. **Finding a pair in a sorted array** – one pointer from start, one from end.
2. **Sliding window / subarray problems** – both pointers move forward to track sums or conditions.
3. **Merging two sorted arrays** – one pointer per array.
4. **Counting / frequency in range** – avoid nested loops.

Tip: Two pointers are mainly used to **avoid O(n²)** brute-force

---

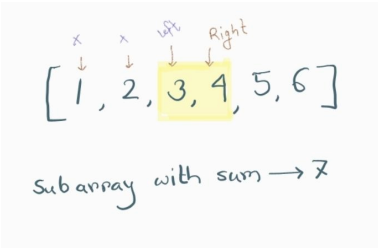
4. Sliding Window Pattern ⭐

```
for (int right = 0; right < n; right++) {
    sum += arr[right];   // expand window

    while (sum > K) {    // shrink window if sum too big
        sum -= arr[left];
        left++;
    }

    maxLength = Math.max(maxLength, right - left + 1);
}
```

Why needed



Grow & shrink subarray efficiently.

---

5. HashMap Basics (NEW & CRITICAL) ⭐⭐⭐

```
HashMap<Integer, Integer> map = new HashMap<>();

map.put(5, 10);
map.put(5, 12);

int v = map.get(5);
int val = map.getOrDefault(3, 0);

Boolean a= map.containsKey(5)

map.remove(5);

int distinct = map.size();
```

// create empty map

// add key=5; value 10
// update key=5 to value 12

// get **value** of key 5, null if key missing
// if 3 not in map, val = 0

// check if key 5 exists

// remove key 5 completely

// number of keys in map

Why needed

```
Key       Value
------    ------
"apple"     5
"mango"     10
"banana"    7
```

- Store both values
- Similar to creating a class and storing them on an arraylist. But this one is faster

---

6. Binary Search Template ⭐⭐

```
while (left <= right) {
       int mid = left + (right - left) / 2;

            if(arr[mid] == target)   return mid;
            else if(arr[mid] < target)   left = mid + 1;
            else    right = mid - 1;
        }

        return -1;
    }
```

Why needed

Fast searching in sorted arrays.

---

7. Lower Bound / Upper Bound Functions ⭐⭐⭐

What to know (idea)

- **Lower Bound** → The first index where the value is **greater than or equal** to a target.
- **Upper Bound** → The first index where the value is **strictly greater** than a target.

Used to count occurrences.

Typical formula:

```
count = upperBound(y) - lowerBound(x)
—————————————————————————
Index:  0  1  2  3  4
Value:  1  3  3  5  7
Target: 3

upperBound = 3
lowerBound = 1

count = 3 - 1 = 2
```

```
—----------------------
```

```
int left=0;
int right= arr.length;

       while(left<right){
           int mid= left + (right-left)/2;


           if(arr[mid]>given) right=mid;
           else if(arr[mid]<given) left= mid+1;

           else if(arr[mid]==given && s.equals("lower")) right=mid;
           else if(arr[mid]==given && s.equals("upper")) left= mid+1;
       }
```

---

8. Subarray Concept (Important Clarification) ⭐

What to know

DO NOT create new arrays.

Represent by indices:

```
length = right - left + 1;
```

More topics needed (Covered in Lab 1):

- Imports / Packages
- Input / Output (BufferedReader, PrintWriter)
- String → Number Parsing (int, long, double)

- StringTokenizer Usage
- String Splitting (.split)
- Collections.sort()
- Comparator Concept (sorting logic)
- Character Utility Methods
- String Helper Methods
- ArrayList Basics & Operations
- Storing Pairs (int[])
- Loops (standard & enhanced for-loop)
- Math Helper Methods

Assignment

A. Two Sum Trouble

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: open, file, creat(, fstream, thread, process, system(, exec(

Your little brother, Bob, loves playing with integers. One day, his teacher gave him a sorted list of ***N*** integers in **non-decreasing** order. Now, your brother wants to play a game with you.

Bob will give you an integer ***S***. You have to find if it is possible to find two values from the list (at distinct positions) whose sum is equal to ***S***.

Since you are feeling very tired, you decide to write a program that can quickly answer Bob's query.

Examples

input Copy

```
4 10
1 3 5 7
```

output Copy

```
2 4
```

input Copy

```
6 18
1 5 8 9 9 10
```

output Copy

```
3 6
```

input Copy

```
4 7
2 4 6 8
```

output Copy

```
-1
```

In the second sample input, 4 5 is also a valid output.

```
import java.util.*;
import java.io.*;

public class A{
    public static void main(String[] args) throws IOException{
        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out= new PrintWriter(System.out);

        //input
        String in=br.readLine();
        StringTokenizer st_in=new StringTokenizer(in, " ");
        int n=Integer.parseInt(st_in.nextToken());
        long sum=Long.parseLong(st_in.nextToken());

        //Filling the array
        String s=br.readLine();
        StringTokenizer st=new StringTokenizer(s, " ");
        long[] arr=new long[n];
```

```
        for(int i=0; i<n; i++){
            arr[i]=Long.parseLong(st.nextToken());
        }


        //Finding the sum
        // 2 pointer used as already sorted
        int left=0;
        int right= n-1;
        boolean found=false;

        while(left<right && found==false){
            long sum_check= arr[left] + arr[right];

            if(sum_check==sum){
                found=true;
                out.println((left+1)+ " "+ (right+1));       //1 based indexing
            }
            else if(sum_check<sum) left++;
            else if(sum_check>sum) right--;
        }

        if(found==false) out.println("-1");
        out.flush();
    }
}
```

B. Two Sum Revisited

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: open, file, creat(, fstream, thread, process, system(, exec(

You are given two integer arrays A and B of sizes N and M, respectively, and an integer K. Both arrays are sorted in non-decreasing order. Your task is to find any pair of indices (i, j) such that:

- i is a valid index in array A $(1 \le i \le N)$
- j is a valid index in array B $(1 \le j \le M)$
- the sum A[i] + B[j] is closest to K (i.e., it minimizes |A[i] + B[j] – K|).

Examples

input Copy

```
4 4 0
-5 -2 -1 5
-5 0 1 1
```

output Copy

```
3 4
```

[Target sum r nearest value(closest difference between target sum and obtained sum)]

```
import java.util.*;
import java.io.*;

public class B{
    public static void main(String[] args) throws IOException{
        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out= new PrintWriter(System.out);

        //input line 1
        String in=br.readLine();
        StringTokenizer st_in=new StringTokenizer(in, " ");
        int n=Integer.parseInt(st_in.nextToken());
        int m=Integer.parseInt(st_in.nextToken());
        long k=Long.parseLong(st_in.nextToken());

        //Filling array 1
        String s1=br.readLine();
        StringTokenizer st1=new StringTokenizer(s1, " ");
        long[] arr1=new long[n];

        for(int i=0; i<n; i++){
            arr1[i]=Long.parseLong(st1.nextToken());
        }
```

```
        //Filling array 2
        String s2=br.readLine();
        StringTokenizer st2=new StringTokenizer(s2, " ");
        long[] arr2=new long[m];

        for(int i=0; i<m; i++){
            arr2[i]=Long.parseLong(st2.nextToken());
        }



        //Near sum
        int left=0;
        int right=m-1;
        long nearest_diff=Long.MAX_VALUE;
        int best_left=0;
        int best_right=m-1;

        while(left<n && right>=0){            //Different condition than normal two pointer
            long sum=arr1[left]+ arr2[right];
            long diff=Math.abs(sum-k);

            if(diff<nearest_diff){
                nearest_diff=diff;
                best_left=left;
                best_right=right;
            }

            if(sum>k) right--;
            if(sum <= k) left++;
        }

        out.println((best_left+1)+ " "+ (best_right+1));
        out.flush();
    }
}
```

******Two pointers won't work in this one******

C. Triple The Trouble

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: open, file, creat(, fstream, thread, process, system(, exec(

You are given an array of n integers, and your task is to find three values (at distinct positions) whose sum is x.

Input

The first input line has two integers n $(1 \le n \le 5000)$ and x $(1 \le x \le 10^9)$, the array size and the target sum. The second line has n integers $a_1, a_2, \ldots, a_n$ $(1 \le a_i \le 10^9)$, the array values.

Output

Print three integers: the positions of the values. If there are several solutions, you may print any of them. If there are no solutions, print -1.

Examples

input Copy

```
7 3
2 1 1 2 2 1 1
```

output Copy

```
2 3 6
```

input Copy

```
3 5
1 3 2
```

output Copy

```
-1
```

```
import java.util.*;
import java.io.*;

public class C {
    public static void main(String[] args) throws IOException{
        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out= new PrintWriter(System.out);

        //Input line 1
        String s_n=br.readLine();
        StringTokenizer st_n= new StringTokenizer(s_n, " ");
        int n= Integer.parseInt(st_n.nextToken());
        long sum= Long.parseLong(st_n.nextToken());

        //Input line 2
        String s=br.readLine();
        StringTokenizer st= new StringTokenizer(s, " ");
        long[] arr=new long[n];
```

```
        for(int i=0; i<n; i++){
            arr[i]= Long.parseLong(st.nextToken());
        }


        //Location
        HashMap<Long, Integer> map=new HashMap<>();
        for(int i=0; i<n; i++){
            map.put(arr[i], i);   //value, index
        }

        //Finding possible sum
        for(int i=0; i<n; i++){
            for(int j=i+1; j<n; j++){
                long more= sum-arr[i]-arr[j];

                if(map.containsKey(more)){
                    int k=map.get(more);

                    if(k!=i && k!=j){
                    out.println((i+1)+ " "+ (j+1)+ " "+ (k+1));
                    out.flush();    //Before return. Has multiple solutions
                    return;        //break would only end the inner loop. Make sure no repeat
                    }
                }
            }
        }
        out.print("-1");
        out.flush();
    }
}
```

D. A Beautiful Sorted List

time limit per test: 1 second
memory limit per test: 1024 megabytes

Forbidden Keywords for the Quiz: sort, open, file, creat(, fstream, thread, process, system(, exec(

Alice and Bob are two friends. Alice has a list of length ***N*** in **non-decreasing** order, and Bob has a list of length ***M***, also in **non-decreasing** order.

Now, they want to combine their lists into a single non-decreasing list of length **N+M**. However, they are not very good at algorithms, so they asked for your help.

Since you are a computer science student, your task is to write an efficient algorithm to merge the two given lists into one non-decreasing list. **Solve the problem in O(N+M)**.

Examples

input Copy

```
4
1 3 5 7
4
2 2 4 8
```

output Copy

```
1 2 2 3 4 5 7 8
```

```
import java.util.*;
import java.io.*;

public class D{
    public static void main(String[] args) throws IOException{
        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out= new PrintWriter(System.out);

        //input 1
        int n=Integer.parseInt(br.readLine());
        String s1=br.readLine();
        StringTokenizer st1=new StringTokenizer(s1, " ");
        long[] arr1=new long[n];

        for(int i=0; i<n; i++){
            arr1[i]=Long.parseLong(st1.nextToken());
        }

        //input 2
        int m=Integer.parseInt(br.readLine());
        String s2=br.readLine();
        StringTokenizer st2=new StringTokenizer(s2, " ");
        long[] arr2=new long[m];
```

```
        for(int i=0; i<m; i++){
            arr2[i]=Long.parseLong(st2.nextToken());
        }



        //Printing sequentially. No need for another array
        int p1=0;
        int p2=0;
        while(p1<n && p2<m){
            if(arr1[p1]<=arr2[p2]){        //Equal to maintain stability
                out.print(arr1[p1]+ " ");
                p1++;
            }

            else if(arr1[p1]>arr2[p2]){
                out.print(arr2[p2]+ " ");
                p2++;
            }
        }


        //Remaining
        while(p1<n) out.print(arr1[p1++]+ " ");
        while(p2<m) out.print(arr2[p2++]+ " ");

        out.flush();
    }
}
```

E. Longest Subarray Sum

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: open, file, creat(, fstream, thread, process, system(, exec(

You are given an array of ***N*** integers and an integer ***K***. Your task is to find the length of the longest contiguous subarray whose sum is less than or equal to ***K***.

Input

The first line contains two integers ***N*** $(1 \le N \le 10^5)$ and ***K*** $(1 \le K \le 10^9)$ — the size of the array and the maximum allowed sum.

The second line contains N space-separated integers $a_1, a_2, a_3 \ldots a_n$ $(1 \le a_i \le 10^6)$ — the elements of the array.

Examples

input Copy

```
5 4
4 1 2 1 5
```

output Copy

```
3
```

input Copy

```
5 5
1 1 1 1 1
```

output Copy

```
5
```

input Copy

```
3 1
2 3 4
```

output Copy

```
0
```

Note:

First example: possible subarrays with sum less than or equal to 4 are [4],[1],[2],[1],[1,2],[2,1],[1,2,1]. Among them, the longest size is 3.

Second example: the sum of the entire array is 5. Hence, we can take the whole array.

Third example: no subarray has sum less than or equal to 1. Hence, the answer is 0.

```
import java.util.*;
import java.io.*;

public class E {
    public static void main(String[] args) throws IOException{
        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out= new PrintWriter(System.out);

        //input
        StringTokenizer n_st=new StringTokenizer(br.readLine(), " ");
        int n=Integer.parseInt(n_st.nextToken());
        int target= Integer.parseInt(n_st.nextToken());
        String s1=br.readLine();
        StringTokenizer st1=new StringTokenizer(s1, " ");
        long[] arr=new long[n];

        for(int i=0; i<n; i++){
            arr[i]=Long.parseLong(st1.nextToken());
        }


        //Finding the longest subarray
        int left=0;
        long sum=0;
        int maxLength=0;

        for(int right=0; right<n; right++){
```

```
            sum= sum + arr[right];

            while(sum>target && left<=right){
                sum= sum - arr[left];
                left++;
            }
            maxLength= Math.max(maxLength, (right-left+1));     //Careful
        }


        out.println(maxLength);
        out.flush();
    }
}
```

F. Longest K-Distinct Subarray

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: open, file, creat(, fstream, thread, process, system(, exec(

You are given an array of integers of length N and an integer K. Your task is to find the length of the longest contiguous subarray that contains at most K distinct elements.

Examples

input Copy

```
4 1
2 1 2 4
```

output Copy

```
1
```

input Copy

```
6 2
6 6 5 6 1 2
```

output Copy

```
4
```

input Copy

```
1 1
1
```

output Copy

```
1
```

```
import java.util.*;
import java.io.*;

public class F {
```

```
    public static void main(String[] args) throws IOException{
        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out= new PrintWriter(System.out);

        //input
        StringTokenizer n_st=new StringTokenizer(br.readLine(), " ");
        int n=Integer.parseInt(n_st.nextToken());
        int k= Integer.parseInt(n_st.nextToken());
        String s1=br.readLine();
        StringTokenizer st1=new StringTokenizer(s1, " ");
        int[] arr=new int[n];

        for(int i=0; i<n; i++){
            arr[i]=Integer.parseInt(st1.nextToken());
        }


        //Finding the longest subarray
        int l=0;
        int maxLength=0;
        int distinct=0;
        HashMap<Integer, Integer> map=new HashMap<>();

        for(int r=0; r<n; r++){
            map.put(arr[r], map.getOrDefault(arr[r],0) + 1);
            if(map.get(arr[r])==1) distinct++;     //Notun add

            while(distinct>k && l<=r){
                map.put(arr[l], map.getOrDefault(arr[left],0) -1);
                if(map.get(arr[l])==0) distinct--;
                l++;
            }

            maxLength= Math.max(maxLength, (right-left+1));    //Don't forget
        }


        out.println(maxLength);
        out.flush();
    }
}
```

G. Count the Numbers

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: open, file, creat(, fstream, thread, process, system(, exec(

You are given a sorted array a of n elements, and some queries. In each query, you are given a pair $[x, y]$ and you have to count how many numbers a_i are there such that $x \leq a_i \leq y$. For example, if the array is $[10, 20, 20, 45, 79]$ and you are given a query $[20, 50]$, then answer will be 3 because there are in total 3 numbers that's value is between 20 and 50.

Example

input Copy

```
5 3
10 20 20 45 79
20 50
5 45
1 100
```

output Copy

```
3
4
5
```

```
import java.util.*;
import java.io.*;

public class G {
    public static void main(String[] args) throws IOException{
        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out= new PrintWriter(System.out);

        //input
        StringTokenizer n_st=new StringTokenizer(br.readLine(), " ");
        int n1=Integer.parseInt(n_st.nextToken());
        int n2= Integer.parseInt(n_st.nextToken());
        String s1=br.readLine();
        StringTokenizer st1=new StringTokenizer(s1, " ");
        int[] arr=new int[n1];

        for(int i=0; i<n1; i++){
            arr[i]=Integer.parseInt(st1.nextToken());
        }


        for(int i=0; i<n2; i++){
            String a=br.readLine();
            StringTokenizer st_a=new StringTokenizer(a, " ");

            long low_given=Long.parseLong(st_a.nextToken());
```

```
            long up_given=Long.parseLong(st_a.nextToken());

            int up_indx= Bound(arr, up_given, "upper");
            int low_indx= Bound(arr, low_given, "lower");

            out.println(up_indx - low_indx);            //No +1
        }

        out.flush();
    }



//helper
    public static int Bound(int[] arr, long given, String s){
        int left=0;
        int right= arr.length-1;

        while(left <= right){
            int mid= left + (right-left)/2;


            if(arr[mid]>given) right=mid-1;
            else if(arr[mid]<given) left= mid+1;

            else if(arr[mid]==given && s.equals("lower")) right=mid-1;
            else if(arr[mid]==given && s.equals("upper")) left= mid+1;
        }
        return left;
    }
}
```

H. Searching is Fun

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: open, file, creat(, fstream, thread, process, system(, exec(

You are given two positive integers: k and x. Consider the sequence of all positive integers that are not divisible by x. Your task is to find the k-th number in this sequence.

For example, if $x = 3$, and $k = 7$, then all numbers that are not divisible by 3 are: $1, 2, 4, 5, 7, 8, 10, 11, 13 \ldots$. The 7-th number among them is 10.

Example

input Copy

```
6
7 3
100 5
49 13
36 2
1 2
11 11
```

output Copy

```
10
124
53
71
1
12
```

```
skipped= (k - 1) / (x - 1)
answer = k + skipped
```

---

k = 7, x = 3

Numbers not divisible by 3:

```
1, 2, 4, 5, 7, 8, 10 ...
```

Compute:

```
skipped = (7 - 1) / (3 - 1) = 6 / 2 = 3
answer = 7 + 3 = 10 ✅
```

```
import java.util.*;
import java.io.*;

public class H {
```

```
    public static void main(String[] args) throws IOException{
        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out= new PrintWriter(System.out);

        int n=Integer.parseInt(br.readLine());
        for(int i=0; i<n; i++){
            StringTokenizer st=new StringTokenizer(br.readLine(), " ");
            long k=Long.parseLong(st.nextToken());
            long x=Long.parseLong(st.nextToken());

            long skipped= (k-1)/(x-1);
            long val= k+ skipped;

            out.println(val);
        }

        out.flush();
    }

}
```